---

author: "Ахтар Хафиз Умер"

group: "НКАБД-07-25"

lang: ru-RU

toc-title: "Содержание"

Отчет по лабораторной работе № 9

Дисциплина: Архитектура компьютера

Студент: Ахтар Хафиз Умер

Группа: НКАБД-07-25

Москва, 2025

---

1. Цель работы

Основной целью данной лабораторной работы является приобретение практических навыков программирования на языке ассемблера NASM с использованием подпрограмм и освоение методов отладки с помощью отладчика GDB.

Конкретные задачи:

- Изучение механизма вызова и возврата из подпрограмм

- Освоение инструкций `call` и `ret` для работы с подпрограммами

- Практическое применение вложенных подпрограмм

- Изучение основных возможностей отладчика GDB

- Освоение методов установки точек останова

- Изучение команд пошагового выполнения и анализа данных

- Разработка навыков поиска и исправления ошибок в программах

---

2. Теоретическое введение

2.1 Понятие подпрограммы

Подпрограмма — это функционально законченный участок кода, который можно многократно вызывать из разных мест программы. В отличие от простых переходов, из подпрограмм существует возврат на команду, следующую за вызовом.

Ключевые инструкции:

| Инструкция | Описание |

|---|---|

| `call метка` | Вызов подпрограммы (сохраняет адрес возврата в стек) |

| `ret` | Возврат из подпрограммы (извлекает адрес возврата из стека) |

2.2 Механизм работы подпрограмм

При вызове подпрограммы:

1. Адрес следующей инструкции сохраняется в стеке

2. Управление передается на первую инструкцию подпрограммы

3. По завершении подпрограммы адрес возврата извлекается из стека

4. Выполнение продолжается с инструкции, следующей за `call`

2.3 Основные возможности отладчика GDB

GDB (GNU Debugger) — мощный отладчик, предоставляющий следующие возможности:

- Установка точек останова (breakpoints)

- Пошаговое выполнение программы

- Просмотр и изменение регистров и памяти

- Дизассемблирование кода

- Анализ стека вызовов

Основные типы точек останова:

- **Breakpoint** - остановка при достижении определенной строки или адреса

- **Watchpoint** - остановка при изменении значения переменной

---

3. Выполнение лабораторной работы

3.1 Программа lab09-1.asm - Базовая работа с подпрограммами

Задача: Создание программы вычисления выражения `f(x) = 2x + 7` с использованием подпрограммы.

Исходный код:

```asm
; lab09-1.asm
; Программа вычисления f(x) = 2x + 7

%include 'in_out.asm'

SECTION .data
    msg: DB 'Введите x: ',0
    result: DB '2x+7=',0

SECTION .bss
    x: RESB 80
    res: RESB 80

SECTION .text
    GLOBAL _start

_start:
    mov eax, msg
    call sprint
    mov ecx, x
    mov edx, 80
    call sread
    mov eax, x
    call atoi
```

```
    call _calcul
    mov eax, result
    call sprint
    mov eax, [res]
    call iprintLF
    call quit

_calcul:
    mov ebx, 2
    mul ebx
    add eax, 7
    mov [res], eax
    ret
```

Результат выполнения:

```bash
@uakhafiz → ~/work/arch-pc/lab09$ ./lab09-1
Введите x: 5
2x+7=17
```

3.2 Модификация программы с вложенными подпрограммами

Задача: Модификация программы для вычисления сложной функции f(g(x)), где:

· g(x) = 3x - 1

· f(x) = 2x + 7

Модифицированный код:

```asm
; lab09-1.asm (модифицированный)
; Программа вычисления f(g(x)) = 2*(3x-1)+7

%include 'in_out.asm'

SECTION .data
    msg: DB 'Введите x: ',0
    result: DB 'f(g(x))=2*(3x-1)+7=',0

SECTION .bss
    x: RESB 80
    res: RESB 80
    temp: RESB 80

SECTION .text
    GLOBAL _start
```

```
_start:
    ; Ввод x
    mov eax, msg
    call sprint
    mov ecx, x
    mov edx, 80
    call sread
    mov eax, x
    call atoi

    ; Вычисление g(x) = 3x - 1
    call _g

    ; Сохранение результата g(x)
    mov [temp], eax

    ; Вычисление f(g(x)) = 2 * g(x) + 7
    call _f

    ; Вывод результата
    mov eax, result
    call sprint
    mov eax, [res]
    call iprintLF
    call quit
```

```
_g:
    mov ebx, 3
    mul ebx
    sub eax, 1
    ret

_f:
    mov ebx, 2
    mul ebx
    add eax, 7
    mov [res], eax
    ret
```

Результат выполнения:

```bash
@uakhafiz → ~/work/arch-pc/lab09$ ./lab09-1
Введите x: 2
f(g(x))=2*(3x-1)+7=17
```

Верификация вычислений:

· g(2) = 3×2 - 1 = 5

· f(5) = 2×5 + 7 = 17 ✓

3.3 Работа с отладчиком GDB

3.3.1 Базовая настройка и запуск

Компиляция с отладочной информацией:

```bash
@uakhafiz → ~/work/arch-pc/lab09$ nasm -f elf -g lab09-2.asm
@uakhafiz → ~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-2 lab09-2.o
```

Запуск GDB:

```bash
@uakhafiz → ~/work/arch-pc/lab09$ gdb ./lab09-2
(gdb) run
Starting program: /home/uakhafiz/work/arch-pc/lab09/lab09-2
Hello, world!
```

3.3.2 Дизассемблирование кода

Синтаксис AT&T:

```asm
mov $0x4,%eax
mov $0x1,%ebx
```

Команда дизассемблирования:

```bash
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>: mov $0x4,%eax
  0x08049005 <+5>: mov $0x1,%ebx
  0x0804900a <+10>: mov $0x804a000,%ecx
  0x0804900f <+15>: mov $0x0,%edx
  0x08049014 <+20>: int $0x80
  0x08049016 <+22>: mov $0x4,%eax
  0x0804901b <+27>: mov $0x1,%ebx
  0x08049020 <+32>: mov $0x804a008,%ecx
  0x08049025 <+37>: mov $0x7,%edx
  0x0804902a <+42>: int $0x80
```

Синтаксис Intel:

```bash
(gdb) set disassembly-flavor intel
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>: mov eax,0x4
  0x08049005 <+5>: mov ebx,0x1
  0x0804900a <+10>: mov ecx,0x804a000
  0x0804900f <+15>: mov edx,0x0
  0x08049014 <+20>: int 0x80
```

3.3.3 Установка точек останова

```bash
(gdb) break _start            # По имени метки
Breakpoint 1 at 0x8049000

(gdb) break *0x8049031         # По адресу инструкции
Breakpoint 2 at 0x8049031

(gdb) info breakpoints         # Просмотр точек останова
Num     Type           Disp Enb Address    What
1       breakpoint     keep y   0x08049000 <_start>
2       breakpoint     keep y   0x08049031
```

```

### 3.3.4 Анализ и модификация данных

Команды работы с данными:

```bash
(gdb) info registers          # Просмотр регистров
(gdb) x/1sb &msg1             # Просмотр строки в памяти
(gdb) set {char}0x804a000='h'   # Изменение символа в памяти
(gdb) print/x $edx            # Вывод регистра в hex-формате
(gdb) print/t $edx            # Вывод регистра в двоичном формате
(gdb) print/d $edx            # Вывод регистра в десятичном формате
```

Пример просмотра и изменения строки:

```bash
(gdb) x/1sb &msg1
0x804a000 <msg1>: "Hello, "
(gdb) set {char}0x804a000='h'
(gdb) x/1sb &msg1
0x804a000 <msg1>: "hello, "
```
```

3.4 Анализ аргументов командной строки в GDB

Загрузка программы с аргументами:

```bash
@uakhafiz → ~/work/arch-pc/lab09$ gdb --args ./lab09-3 argument1 argument2 'argument 3'
```

Анализ стека аргументов:

```bash
(gdb) break _start
(gdb) run
(gdb) x/4xw $esp
0xbffff400: 0x00000005  0x08049000  0xbffff500  0xbffff510
```

Структура стека аргументов:

Адрес Значение

$esp Количество аргументов (argc)

$esp+4 Адрес имени программы

$esp+8 Адрес первого аргумента

$esp+12 Адрес второго аргумента

Наблюдения: Аргументы в кавычках сохраняются как единое целое, аргументы с пробелами разделяются.

---

4. Выполнение самостоятельной работы

4.1 Поиск и исправление ошибки в программе

Исходная программа: Вычисление выражения (3 + 2) × 4 + 5

Обнаруженная проблема: Программа выдавала неверный результат (10 вместо 25)

```bash
@uakhafiz → ~/work/arch-pc/lab09$ ./lab09-4
Результат: 10
```

Анализ в GDB:

```asm
mov ebx, 3          ; ebx = 3
mov eax, 2          ; eax = 2
add ebx, eax        ; ebx = 5 (3+2)
mov ecx, 4          ; ecx = 4
mul ecx             ; eax = 2 × 4 = 8 (ОШИБКА!)
```

```
add ebx, 5          ; ebx = 5 + 5 = 10
```

Выявленная ошибка: Инструкция mul ecx умножает регистр eax на ecx, но в eax находится значение 2, а не результат сложения (5).

4.2 Исправленная версия программы

Исправленный код:

```asm
; lab09-4.asm (исправленный)
; Программа вычисления (3+2)*4+5

%include 'in_out.asm'

SECTION .data
    div: DB 'Результат: ',0

SECTION .text
    GLOBAL _start

_start:
    ; --- Вычисление выражения (3+2)*4+5
    mov eax, 3
    mov ebx, 2
```

```
    add eax, ebx      ; eax = 5 (3+2)
    mov ecx, 4
    mul ecx           ; eax = 5 × 4 = 20
    add eax, 5        ; eax = 20 + 5 = 25
    mov edi, eax      ; Сохранение результата

    ; --- Вывод результата на экран
    mov eax, div
    call sprint
    mov eax, edi
    call iprintLF
    call quit
```

Результат выполнения:

```bash
@uakhafiz → ~/work/arch-pc/lab09$ ./lab09-4
Результат: 25
```

4.3 Верификация исправления

Пошаговая проверка:

1. (3 + 2) = 5

2. 5 × 4 = 20

3. 20 + 5 = 25

Вывод: Программа работает корректно, вычисления верифицированы.

---

5. Выводы

В ходе выполнения лабораторной работы № 9 достигнуты значительные результаты:

Задача Результат

Освоение работы с подпрограммами Изучены механизмы вызова и возврата, практическое применение инструкций call и ret, создание модульной структуры программ

Разработка вложенных подпрограмм Успешно реализована архитектура с несколькими уровнями вложенности подпрограмм, освоены методы передачи параметров между ними

Глубокое изучение отладчика GDB Освоены основные команды отладки, методы установки точек останова, техники пошагового выполнения и анализа состояния программы

Дизассемблирование Изучены различия между синтаксисами AT&T и Intel, освоены методы анализа машинного кода

Модификация данных в памяти Практическое применение команд просмотра и изменения содержимого памяти и регистров в процессе отладки

Анализ аргументов командной строки Изучен механизм передачи аргументов через стек, методы их извлечения и анализа в отладчике

Поиск и исправление ошибок Успешно выявлена и исправлена логическая ошибка в

программе, продемонстрирована эффективность применения инструментов отладки

Полученные навыки составляют важную основу для профессиональной разработки программного обеспечения, обеспечивая глубокое понимание внутренних механизмов выполнения программ и эффективных методов их отладки.

---

6. Приложение. Исходные коды программ

Программа lab09-1.asm

```asm
%include 'in_out.asm'

SECTION .data
    msg: DB 'Введите x: ',0
    result: DB '2x+7=',0

SECTION .bss
    x: RESB 80
    res: RESB 80

SECTION .text
    GLOBAL _start
```

```
_start:
    mov eax, msg
    call sprint
    mov ecx, x
    mov edx, 80
    call sread
    mov eax, x
    call atoi
    call _calcul
    mov eax, result
    call sprint
    mov eax, [res]
    call iprintLF
    call quit

_calcul:
    mov ebx, 2
    mul ebx
    add eax, 7
    mov [res], eax
    ret
```

Программа lab09-2.asm

```asm
SECTION .data
    msg1: db "Hello, ",0
    msg1Len: equ $- msg1
    msg2: db "world!",0xa
    msg2Len: equ $- msg2

SECTION .text
    GLOBAL _start

_start:
    mov eax, 4
    mov ebx, 1
    mov ecx, msg1
    mov edx, msg1Len
    int 0x80

    mov eax, 4
    mov ebx, 1
    mov ecx, msg2
    mov edx, msg2Len
    int 0x80

    mov eax, 1
    mov ebx, 0
```

```
    int 0x80
```

Программа lab09-4.asm (исправленная)

```asm
%include 'in_out.asm'

SECTION .data
    div: DB 'Результат: ',0

SECTION .text
    GLOBAL _start

_start:
    ; --- Вычисление выражения (3+2)*4+5
    mov eax, 3
    mov ebx, 2
    add eax, ebx
    mov ecx, 4
    mul ecx
    add eax, 5
    mov edi, eax

    ; --- Вывод результата на экран
```

```
    mov eax, div
    call sprint
    mov eax, edi
    call iprintLF
    call quit
```

---

7. Основные команды GDB

Команда Описание

gdb ./program Запуск отладчика

gdb --args ./program arg1 arg2 Запуск с аргументами

break _start Установка точки останова

run / r Запуск программы

stepi / si Пошаговое выполнение (вход в подпрограммы)

nexti / ni Пошаговое выполнение (обход подпрограмм)

continue / c Продолжение выполнения

disassemble _start Дизассемблирование кода

info registers Просмотр регистров

print $eax Вывод значения регистра

x/1sb &msg Просмотр строки в памяти

set {char}address='c' Изменение символа в памяти

quit / q Выход из отладчика

```

---

## Files to Create for Lab 9

| File Name | Description |
|----------|----------|
| `lab09-1.asm` | Базовая программа с подпрограммой |
| `lab09-2.asm` | Программа для отладки в GDB |
| `lab09-3.asm` | Обработка аргументов командной строки |
| `lab09-4.asm` | Исправленная программа (самостоятельная работа) |
| `in_out.asm` | Библиотека функций ввода-вывода |

---
```

12:29 PM

[Preview] R...
6g.github.dev

```
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '' >> lab8
-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo 'next:' >>
 lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    cmp e
cx, 0' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    jz _e
nd' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    pop e
ax' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    call
atoi' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    add e
si, eax' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    loop
next' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '' >> lab8
-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '_end:' >>
 lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    mov e
ax, msg' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    call
sprint' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    mov e
ax, esi' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    call
iprintLF' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    call
quit' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ nasm -f elf lab
8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ ld -m elf_i386
lab8-3.o -o lab8-3
@uakhafiz ➜ ~/work/arch-pc/lab08 $ ./lab8-3 10 20
30 40
Сумма: 1000
@uakhafiz ➜ ~/work/arch-pc/lab08 $ cp lab8-1.asm l
ab8-2.asm lab8-3.asm in_out.asm /workspaces/study_
2024-2025_arch-pc/labs/lab08/
@uakhafiz ➜ ~/work/arch-pc/lab08 $ cd /workspaces/
study_2024-2025_arch-pc
@uakhafiz ➜ /workspaces/study_2024-2025_arch-pc (m
ain) $ git add labs/lab08/
@uakhafiz ➜ /workspaces/study_2024-2025_arch-pc (m
ain) $ git commit -m "Add Lab 8 programs"
[main a8f2b71] Add Lab 8 programs
 5 files changed, 245 insertions(+)
 create mode 100644 labs/lab08/in_out.asm
 create mode 100644 labs/lab08/lab8-1.asm
 create mode 100644 labs/lab08/lab8-2.asm
 create mode 100644 labs/lab08/lab8-3.asm
 create mode 100644 labs/lab08/report/report.md
@uakhafiz ➜ /workspaces/study_2024-2025_arch-pc (m
ain) $ git push
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 2 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (10/10), 2.33 KiB | 2.33 MiB
/s, done.
Total 10 (delta 1), reused 0 (delta 0), pack-reuse
d 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed wi
th 1 local object.
To https://github.com/uakhafiz/study_2024-2025_arc
h-pc
   15bf391..a8f2b71  main -> main
@uakhafiz ➜ /workspaces/study_2024-2025_arch-pc (m
ain) $ mkdir -p /workspaces/study_2024-2025_arch-p
c/labs/lab09/report
@uakhafiz ➜ /workspaces/study_2024-2025_arch-pc (m
ain) $ cd /workspaces/study_2024-2025_arch-pc/labs
/lab09/report
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ pwd
/workspaces/study_2024-2025_arch-pc/labs/lab09/rep
ort
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo '---' > report.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo 'title: "Лабораторная работа
 №9"' >> report.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo 'author: "Ахтар Хафиз Умер"'
 >> report.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo 'group: "НКАбд-07-25"' >> re
port.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo 'lang: ru-RU' >> report.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo 'toc-title: "Содержание"' >>
 report.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo '---' >> report.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo '' >> report.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo '# Лабораторная работа №9' >
> report.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo '# Понятие подпрограммы. Отл
адчик GDB.' >> report.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $
```

12:30 PM

[Preview] R...
6g.github.dev

```
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '_end:' >>
 lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    mov e
ax, msg' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    call
sprint' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    mov e
ax, esi' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    call
iprintLF' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    call
quit' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ nasm -f elf lab
8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ ld -m elf_i386
lab8-3.o -o lab8-3
@uakhafiz ➜ ~/work/arch-pc/lab08 $ ./lab8-3 10 20
30 40
Сумма: 1000
@uakhafiz ➜ ~/work/arch-pc/lab08 $ cp lab8-1.asm l
ab8-2.asm lab8-3.asm in_out.asm /workspaces/study_
2024-2025_arch-pc/labs/lab08/
@uakhafiz ➜ ~/work/arch-pc/lab08 $ cd /workspaces/
study_2024-2025_arch-pc
@uakhafiz ➜ /workspaces/study_2024-2025_arch-pc (m
ain) $ git add labs/lab08/
@uakhafiz ➜ /workspaces/study_2024-2025_arch-pc (m
ain) $ git commit -m "Add Lab 8 programs"
[main a8f2b71] Add Lab 8 programs
 5 files changed, 245 insertions(+)
 create mode 100644 labs/lab08/in_out.asm
 create mode 100644 labs/lab08/lab8-1.asm
 create mode 100644 labs/lab08/lab8-2.asm
 create mode 100644 labs/lab08/lab8-3.asm
 create mode 100644 labs/lab08/report/report.md
@uakhafiz ➜ /workspaces/study_2024-2025_arch-pc (m
ain) $ git push
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 2 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (10/10), 2.33 KiB | 2.33 MiB
/s, done.
Total 10 (delta 1), reused 0 (delta 0), pack-reuse
d 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed wi
th 1 local object.
To https://github.com/uakhafiz/study_2024-2025_arc
h-pc
   15bf391..a8f2b71  main -> main
@uakhafiz ➜ /workspaces/study_2024-2025_arch-pc (m
ain) $ mkdir -p /workspaces/study_2024-2025_arch-p
c/labs/lab09/report
@uakhafiz ➜ /workspaces/study_2024-2025_arch-pc (m
ain) $ cd /workspaces/study_2024-2025_arch-pc/labs
/lab09/report
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ pwd
/workspaces/study_2024-2025_arch-pc/labs/lab09/rep
ort
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo '---' > report.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo 'title: "Лабораторная работа
 №9"' >> report.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo 'author: "Ахтар Хафиз Умер"'
 >> report.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo 'group: "НКАбд-07-25"' >> re
port.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo 'lang: ru-RU' >> report.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo 'toc-title: "Содержание"' >>
 report.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo '---' >> report.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo '' >> report.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo '# Лабораторная работа №9' >
> report.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo '# Понятие подпрограммы. Отл
адчик GDB.' >> report.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo '' >> report.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo '## Цель работы' >> report.m
d
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo '' >> report.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ echo 'Приобретение навыков написа
ния программ с использованием подпрограмм. Знакомс
тво с методами отладки при помощи GDB и его основн
ыми возможностями.' >> report.md
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ mkdir -p ~/work/arch-pc/lab09
@uakhafiz ➜ .../study_2024-2025_arch-pc/labs/lab09
/report (main) $ cd ~/work/arch-pc/lab09
@uakhafiz ➜ ~/work/arch-pc/lab09 $ pwd
/home/codespace/work/arch-pc/lab09
@uakhafiz ➜ ~/work/arch-pc/lab09 $
```